

AI Agent 核心概念：Agent Loop、Plan-and-Execute、A2A、Agentic Workflows、Tools 注册

第一次被 ChatGPT 震到的时候，很多人应该都还在研究 Prompt 怎么写。那时候它更像一个会聊天的知识库。你问，它答；你不问，它也不会自己动。三年过去，AI 已经不只是在聊天框里回复文字了。它开始会调用工具，会读文件，会跑代码，甚至能操作电脑界面。

再往前走一步，就是现在大家反复提到的 AI Agent。

OpenAI 有 Assistant API，Anthropic 有 Claude Agent，Coze、Dify 这类低代码平台也都在围绕 Agent 做能力封装。热度确实高，但很多人聊 Agent 时容易把概念讲得特别玄。

这篇会把 AI Agent 拆开讲清楚。全文接近 7000 字，主要看这几块：

1. Agent 是怎么一步步从聊天机器人进化到常驻自治系统的
2. Agent、传统编程、Workflow 到底有什么区别，什么时候该用哪个
3. Agent = LLM + Planning + Memory + Tools 这个公式每一层负责什么
4. ReAct、Plan-and-Execute、Reflection、Multi-Agent 这些范式到底怎么选
5. Agent 面临的真实挑战和落地时的工程选型建议

AI Agent 的演进

AI Agent 不是突然冒出来的。它大概经历了几次明显变化。

2022 年，ChatGPT 这类产品刚火的时候，大家主要还在和模型“对话”。能力很强，但它只能基于已有知识回答问题，不能主动调用外部工具，也不能自己完成操作。

当时最重要的玩法是 Prompt Engineering。你把提示词写得越清楚，它回答得越稳。

但它本质上还是会说，不是会做。

2023 年中，Function Calling 出现后，事情开始变了。

LLM 可以调用外部 API，不再只是生成文字。RAG 也开始大规模应用，AI 有了外部知识库和“外部记忆”。AutoGPT 这类早期 Agent 尝试也在这个阶段出现。

不过早期体验比较粗糙。很多任务跑着跑着就开始绕圈，甚至陷入无限循环。

2023 年底，大家开始重视编排。

ReAct 这种推理框架逐渐被接受。模型不只是直接给答案，而是先思考下一步要做什么，再决定是否调用工具，然后根据工具返回结果继续推理。

多 Agent 协作也开始被讨论。比如一个 Agent 负责规划，一个 Agent 负责执行，一个 Agent 负责检查。

Coze、Dify 这类平台把开发门槛降了下来，用 DAG（有向无环图）来约束执行流程，避免 AutoGPT 那种完全放飞的自治方式。

2024 年底，标准化和多模态开始变重要。

MCP 协议出现，解决工具接入碎片化的问题。Computer Use 让 Agent 可以操作图形界面。

AI 编程工具也在这个阶段快速发展。Cursor、Claude Code、Codex 这类工具把代码库阅读、修改、测试、提交串了起来，“Vibe Coding”也在这个阶段被更多人讨论。

2025 年，Agent 开始往长任务执行方向走。

这一年，Agent 不再只是一次对话里的助手，而是开始变成可以接任务、跑流程、产出结果的工作单元。

Agent Skills 这类机制也开始出现。很多任务不是一句 Prompt 就能做好，而是需要固定流程、上下文、模板、脚本和校验规则。Skill 做的就是把这些方法封装下来，让 Agent 遇到类似任务时按既定流程执行。

到了 2026 年，Agent 开始更接近长期在线的数字工作单元。

OpenClaw 这类项目把 Skills 和 Heartbeat 推到更显眼的位置。

Skills 负责封装能力，Heartbeat 负责周期性唤醒 Agent，让它检查消息、处理任务、更新状态，而不是只能等用户下一次提问。

不过，Heartbeat 不等于真正的连续意识，它更像定时唤醒；本地数据主权也不等于绝对安全。只要 Agent 能安装 Skill、访问文件、执行脚本，就会带来新的权限、沙箱和供应链风险。

Harness Engineering 也开始被更多人讨论。

可以简单理解为：Agent = Model + Harness。模型负责推理和生成，Harness 负责把模型放进一个可执行、可观察、可恢复、可验证的工作环境里。

大家不再只盯着模型参数、上下文长度和 Prompt 技巧，开始更关注模型外面的工程环境。

后续发展展望。

再往后看，几个方向会继续推进：内建记忆、预测能力，以及从数字世界扩展到物理机器人。

不过这个阶段划分，别看得太死。真实产品经常同时具备多个阶段的特征。比较明显的分水岭还是 2023 年中，之前 AI 基本只能“说”，之后才开始逐渐能“做”。

Agent、传统编程和 Workflow 区别？

很多人第一次接触 Agent，会把它和自动化脚本、Workflow 混在一起。

其实可以先看一个最简单的区别：

- | | |
|---|----------------------------|
| 1 | 传统编程：程序员写代码 → 执行结果 |
| 2 | Workflow：产品画流程图 → 执行结果 |
| 3 | Agent：用户说意图 → AI 决策 → 动态执行 |

传统编程适合逻辑固定、高频执行、对性能要求很高的场景。比如订单扣库存、支付状态流转、消息队列消费，这些就别硬上 Agent。

Workflow 适合流程清晰、步骤有限、需要可视化管理的场景。比如审批流、内容发布流、线索分配流，出问题也好排查。

Agent 适合步骤不确定、需要理解自然语言意图、执行中还要动态判断的任务。比如“帮我排查今天早上服务变慢的原因”，这类任务很难提前把每一步都写死。

如果是超长流程，里面又夹杂一些动态子任务，可以用 Plan-and-Execute。它更像 Workflow 和 Agent 的混合体。Agent 解决的是那些没法提前穷举所有情况的问题。Workflow 和传统编程更接近，都是人在提前控制流程，只是一个用代码，一个用图形化流程。

Agent 面临的挑战有哪些？

聊 Agent 不能只讲愿景，也得说点真实问题。

- 长任务跑久了，历史信息会被截断，模型会“失忆”。更烦的是，上下文变长后推理质量不一定更好，很多模型对中间位置的信息利用效率并不高
- 工具调用可以降低幻觉，但不能彻底消灭。LLM 在推理步骤里仍然可能生成错误判断，工具返回结果也不一定能把它拉回来

- 多轮迭代、工具调用、日志回传、上下文压缩，每一项都在烧 Token。复杂任务跑一轮，账单可能真会让人清醒
- Agent 能执行代码、调 API、读写文件，也就一定会面对 Prompt Injection 和越权操作风险。更现实的做法是权限最小化、沙箱隔离、高危操作人工确认
- 深度多步推理任务里，LLM 还是容易局部最优，可能看起来一直在推进，其实已经偏题了
- Agent 为什么做了某个决策、为什么调用了某个工具、是哪一步把上下文带偏了，排查起来很头疼

后面比较确定的方向包括：更长上下文、分层记忆、多模态 GUI 操作、沙箱和权限体系、推理效率优化。

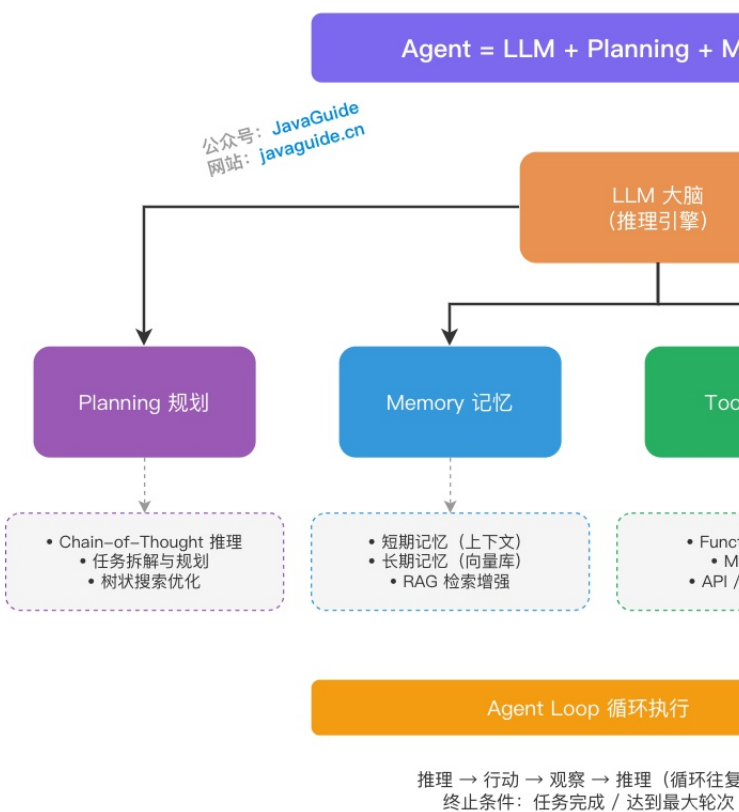
什么是 AI Agent ?

如果你看过 LangChain 的 Agent 源码，会发现它的核心并不神秘，很多时候就是一个 while 循环。

AI Agent 可以理解为一个能感知环境、做决策、执行动作的软件系统。LLM 负责理解和决策，工具负责执行，记忆负责保存上下文和历史经验。

它和普通聊天机器人的差别在于：Agent 不只是回复消息，它会在动态环境里持续观察、判断、执行，直到任务结束。一般可以用这个公式概括：**Agent = LLM + Planning + Memory + Tools**。

AI Agent 核心



AI Agent 核心架构

推理与规划 (Reasoning / Planning)：用 LLM 分析当前任务状态，拆目标，决定下一步怎么做。Chain-of-Thought (CoT) 提示技术可以让模型逐步推理，减少直接拍脑袋给答案的概率。

记忆分两层。短期记忆通常是上下文历史，用来保持对话连续性；长期记忆一般是外部知识库，比如向量数据库或知识图谱。短期记忆解决“刚才说过什么”，长期记忆解决“过去积累了什么”。

Tools (工具)：让 LLM 能真正操作外部世界，比如查数据、调 API、读文件、执行代码。没有工具，Agent 很多时候只能停留在“建议你这么做”。

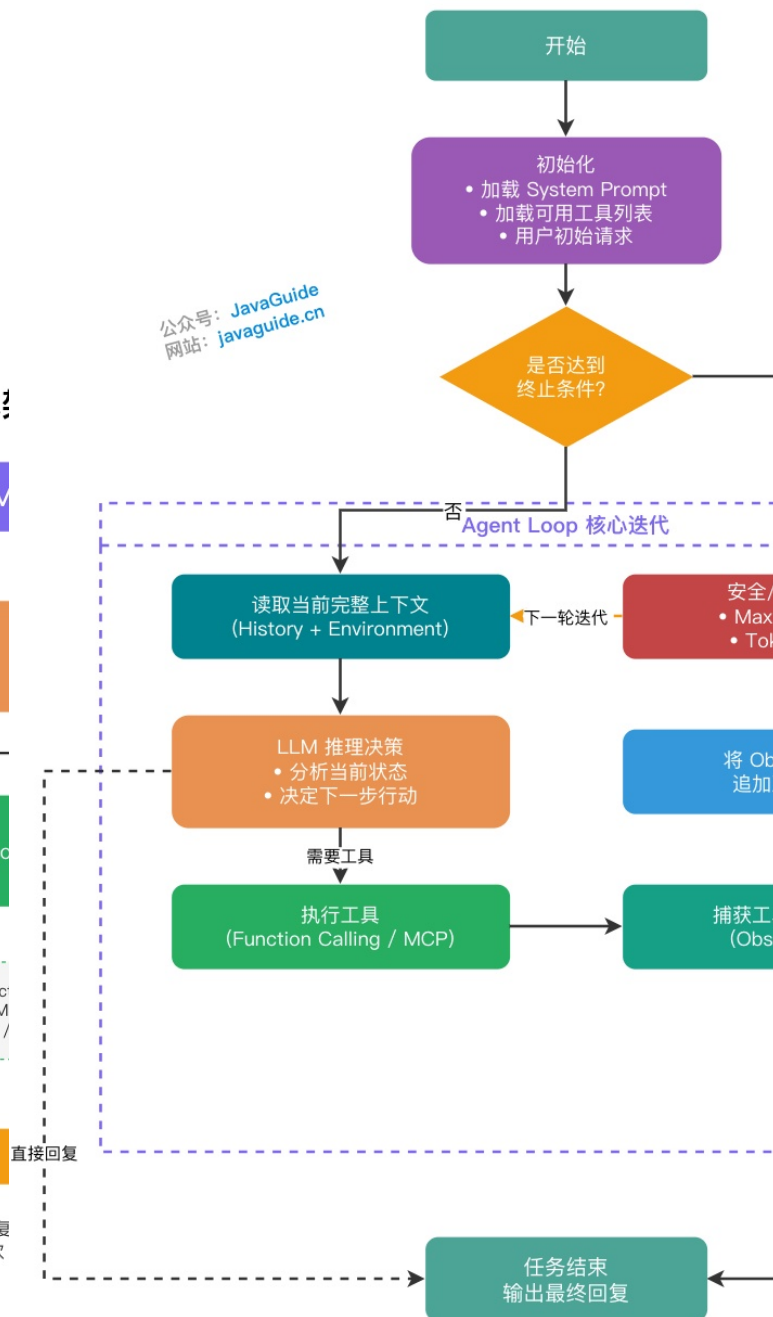
工具执行后会返回结果，Agent 把这些结果放回上下文，再进入下一轮推理。这个反馈闭环就是 Observation (观察)，也是 Agent Loop 能转起来的关键。

什么是 Agent Loop ?

Agent Loop 是 Agent 真正跑起来的地方。

它每一轮大概做三件事：让 LLM 推理，调用工具，把工具结果写回上下文。一直循环，直到任务完成或者触发停止条件。

Agent Loop 工作流程



Agent Loop 工作流程

流程大概是这样：

1. 初始化时加载 System Prompt、可用工具列表、用户初始请求
2. 循环迭代——读取上下文，LLM 推理决定下一步（调用工具还是直接回复），触发并执行工具，捕获返回结果追加到上下文
3. LLM 判断任务完成，不再调用工具时退出循环

- 安全兜底——防止死循环, 设置最大迭代轮次上限(一般 10 到 20 轮) 或 Token 消耗阈值

工程难点不在 while 循环本身, 而在上下文管理。

任务越跑越久, 上下文会越来越长。关键信息被稀释后, 模型就容易跑偏。这也是 Context Engineering 要解决的问题。

LangChain、LlamaIndex、Spring AI 这些框架都对 Agent Loop 做了封装, 但底层思路差不多。

做一个 Agent 系统, 最少要搞定哪三层?

做一个 Agent 系统, 通常绕不开这三层。

- LLM Call**: 这一层负责模型调用。比如 OpenAI、Anthropic、Hugging Face 的接口差异, 流式输出, Token 截断, 重试机制, 都在这里处理。
- Tools Call**: 这一层负责让 LLM 和外部系统交互。Function Calling、MCP、Skills 都可以放在这里看。读写本地文件、网页搜索、代码沙箱、第三方 API 调用, 都属于工具能力。
- Context Engineering**: 这一层负责管理传给大模型的 Prompt 和上下文。狭义看, 它是系统提示词编排。放宽一点, 它还包括动态记忆注入、会话状态管理、工具描述动态组装。

能调模型、能用工具、能管上下文, Agent 的能力栈就基本成型了。

这里最容易被低估的是 Context Engineering。很多模型能力不差, 最后效果不行, 是上下文喂得太乱。不给任何 Context 的情况下, 再先进的模型也可能只能处理极少数任务。

Tools 注册与调用遵循什么标准格式?

Agent 想准确调用外部工具, 绕不开两个东西: OpenAI Schema 和 MCP。

OpenAI Schema 解决数据格式问题, MCP 解决通信接入问题。

数据格式: Function Calling Schema

外部工具可以很复杂, 但 LLM 推理时只认结构化描述。

现在主流的数据格式基本都在向 OpenAI Function Calling Schema 靠拢。Anthropic、Google 这些厂商也都支持类似形式。

它用 JSON Schema 描述工具名称、用途、参数类型、必填字段。模型根据这段描述判断要不要调用工具, 以及参数该怎么填。

比如一个大数据工程师常见的工具: 查询慢 SQL 日志。

```
1 {
2   "type": "function",
3   "function": {
4     "name": "query_slow_sql",
5     "description": "查询指定微服务在特定时间段的慢 SQL 日志。服务响应
    ↳ 慢、数据库超时、CPU 飙升的时候用这个。如果用户问的是网络或内存问题,
    ↳ 别调这个。",
6     "parameters": {
7       "type": "object",
8       "properties": {
9         "service_name": {
10          "type": "string",
11          "description": "服务名, 比如 user-service、order-service"
12        },
13        "time_range": {
14          "type": "string",
15          "description": "时间范围, 格式 HH:MM-HH:MM, 比如
    ↳ 09:00-09:30"
16        },
17        "threshold_ms": {
18          "type": "integer",
19          "description": "慢 SQL 判定阈值 (毫秒), 默认 1000"
20        }
21      },
22      "required": ["service_name", "time_range"]
23    }
24  }
25 }
```

工具描述写得好不好, 会直接影响 Agent 的判断。

模型到底该不该调用这个工具, 应该填哪些参数, 主要都靠 description。好的描述要把使用场景和禁用场景讲清楚。

比如上面那句“如果用户问的是网络或内存问题, 别调这个”, 就很有用。

进阶封装: Skills

有些任务不是调用一个原子工具就能完成的。比如“排查数据库慢查询”, 得先读日志、跑分析脚本、对照团队规范给出建议。如果每次都从零开始, Agent 的输出既不稳定, 也没法复用。

这就是 Skill 要解决的问题。Skill 更像一份可调用的经验包: 把一类任务的执行顺序、约束条件和踩坑记录写下来, 让 Agent 在判断当前任务命中时才把它读进来, 而不是启动就全部塞进上下文。

目前 Skill 有两种主流形态:

1. 传统 Toolkits (黑盒): 把多个原子工具在代码层封装成一个高阶工具, 对外只暴露 JSON Schema, LLM 看不到内部执行路径。推理步骤少、Token 消耗低, 适合逻辑固定的场景。

2. Agent Skills (白盒): 以 SKILL.md 为核心的自然语言指令集。每个 Skill 是一个独立文件夹:

```
1 .claude/skills/code-reviewer/
2 SKILL.md           ↳ YAML front-matter + 详细指令
3 scripts/xxx.py      ↳ 可选: 配套脚本
4 reference.md        ↳ 可选: 参考资料
```

SKILL.md 分两部分: 前面是轻量元数据, 告诉宿主“我是谁、什么时候该用我”; 后面是正文, 写具体流程、约束和示例。启动时只读元数据做发现, 等 LLM 判断需要某个 Skill, 再把完整正文加载进上下文。这种延迟加载设计, 是 Agent Skills 和传统 Toolkits 最大的不同。

Claude Code、Cursor 这类工具已经原生支持这套模式, 会自动扫描项目里的 .claude/skills/ 目录, 由模型自己判断哪个 Skill 该激活。

纯代码封装、调用路径固定, 用 Toolkits。团队经验沉淀、任务流程灵活, 用 Agent Skills 更合适。更详细的 Skills 工程实践——包括路由设计、SKILL.md 写法避坑、第三方 Skill 安全审计, 可以看:《Agent Skills 详解》。

通信接入: MCP 协议

Function Calling Schema 让模型知道工具“长什么样”。

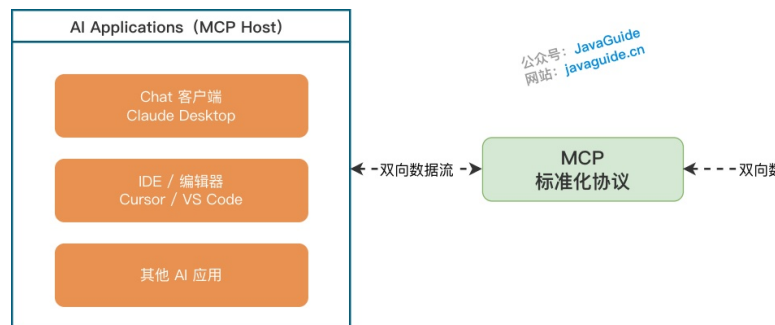
MCP 解决的是另一个问题: 工具怎么接入宿主程序。

Anthropic 在 2024 年 11 月推出 MCP。它要解决的痛点很直接: 以前开发者要在代码里手动维护一堆映射, 比如:

工具名称 → 实际执行函数 + JSON Schema 描述

接一个新工具, 就写一堆胶水代码。工具越多, 维护越难。

MCP 提供了一套基于 JSON-RPC 2.0 的统一通信协议, 经常被叫作 AI 领域的“USB-C 接口”。外部系统通过 MCP Server 暴露能力, 宿主程序连接 Server 后, 就能自动发现并注册工具。



MCP 图解

这样 AI 应用和底层外部代码就解耦了。

MCP 定义了三类标准原语:

原语类型	作用	例子
Tools	LLM 主动调用的函数	查询数据库、发送邮件、执行代码
Resources	Agent 按需读取的只读数据	本地文件、数据库记录、日志流
Prompts	可复用的提示词模板	代码审查模板、故障报告模板

这里容易混的一点是：MCP Server 对外暴露工具时，内部还是会用 JSON Schema 描述参数规范。

JSON Schema 是数据格式，MCP 是通信协议层。

什么是 Prompt Engineering ?

Prompt（提示词）可以简单理解为给大语言模型下达的指令。Prompt Engineering 就是怎么把这条指令写清楚，让模型输出更可控。关键在边界是否清晰——指令越模糊，模型越容易乱猜；指令越结构化，输出就越稳定。

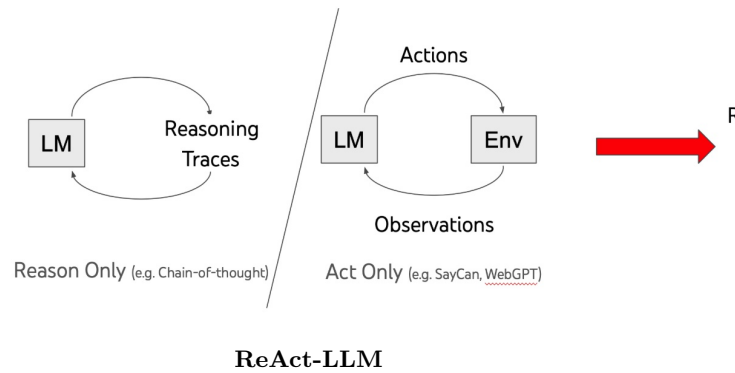
这块展开讲内容很多，可以单独看这篇：《提示词工程（Prompt Engineering）》。

什么是 Context Engineering ?

很多时候，Agent 做不好，不是模型能力太多，而是上下文太乱。

Context Engineering 做的事情，就是在有限 Token 窗口里，把最有用的信息喂给模型，把噪声挡在外面。它很容易和 Prompt Engineering 混在一起。

Prompt Engineering 更偏提示词怎么写，Context Engineering 管得更宽，包括规则、记忆、工具描述、会话状态、外部观察结果、Token 预算。



比如任务是：帮我排查一下今天早上 user-service 接口变慢的原因，并把结果发给负责人。

ReAct 跑起来大概是这样。

它先查 user-service 早上的监控，发现 9 点到 9:30 CPU 飙升到 98%，同时有大量慢 SQL 告警。

然后顺着这条线去翻日志，捞出那条慢 SQL，发现是一个没走索引的表扫描。

接着去查服务负责人，通讯录里找到王建国，邮箱是 wangjianguo@company.com。

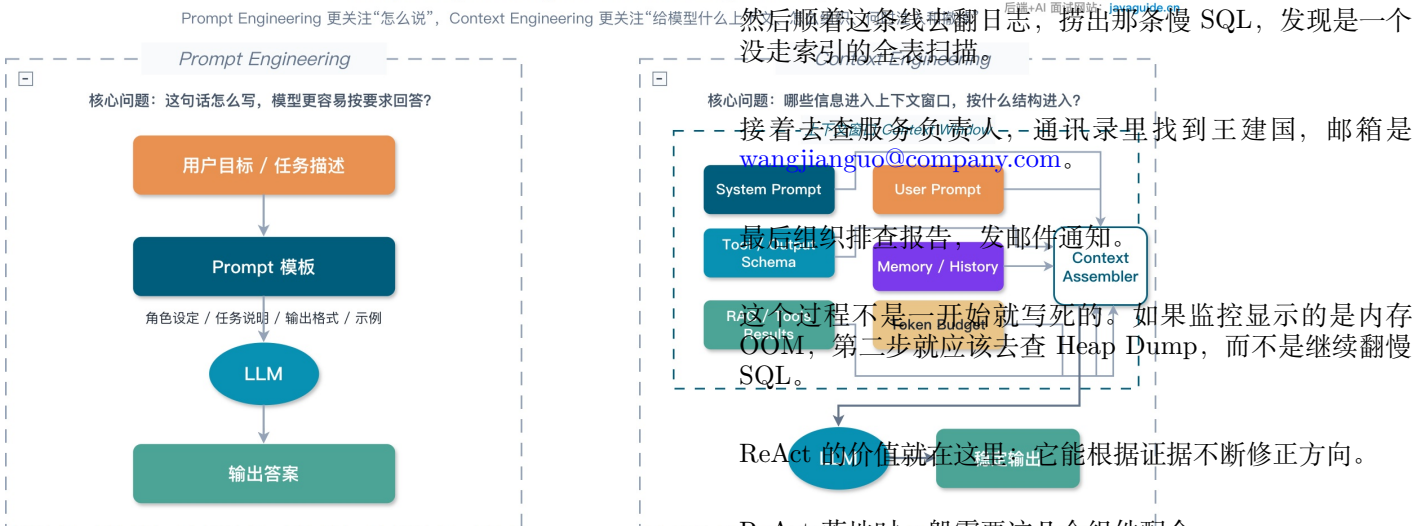
最后组织排查报告，发邮件通知。这个过程不是一开始就写死的。如果监控显示的是内存 OOM，第二步就应该去查 Heap Dump，而不是继续翻慢 SQL。

ReAct 的价值就在这里：它能根据证据不断修正方向。

ReAct 落地时一般需要这几个组件配合：

1. 历史上下文，保存推理步骤、执行动作、反馈观察
2. 实时环境输入，比如系统告警、用户反馈等外部变量
3. LLM 推理模块：负责逻辑分析和下一步规划
4. 工具集与技能库，包括原子工具和 Skills
5. 反馈观察机制，采集工具响应并追加回上下文

Prompt Engineering vs Context Engineering



Context Engineering 和 Prompt Engineering 差别

这块展开讲内容很多，可以单独看这篇：《提示词工程（Prompt Engineering）》和《上下文工程（Context Engineering）》。

Agent 核心范式有哪些？

ReAct

ReAct 是 Reasoning + Acting，由 Shunyu Yao 等人在 2022 年提出，论文是《ReAct: Synergizing Reasoning and Acting in Language Models》。

LangChain、LlamaIndex、AgentScope 这类框架里的 Agent 模块，很多都能看到这个范式的影子。

它的思路很直观：模型先推理一步，拿到外部环境反馈，再推理下一步，交替进行。

LLM 自己容易缺少实时信息，也容易幻觉。ReAct 就让它“走一步看一步”，每一步都根据工具返回结果继续判断。

ReAct 模式流程 (Reasoning + Acting)

遇到失败后，模型反思出“变量 count 在调用前没初始化”，下一轮就能规避。

Self-Refine 方法：任务完成后，让模型审查自己的输出，再迭代改进。它通常用来提升回答、代码、文案这类输出质量。

CRITIC 方法：引入外部工具，比如搜索引擎或代码执行器，对输出做事实验证，再根据验证结果修正。

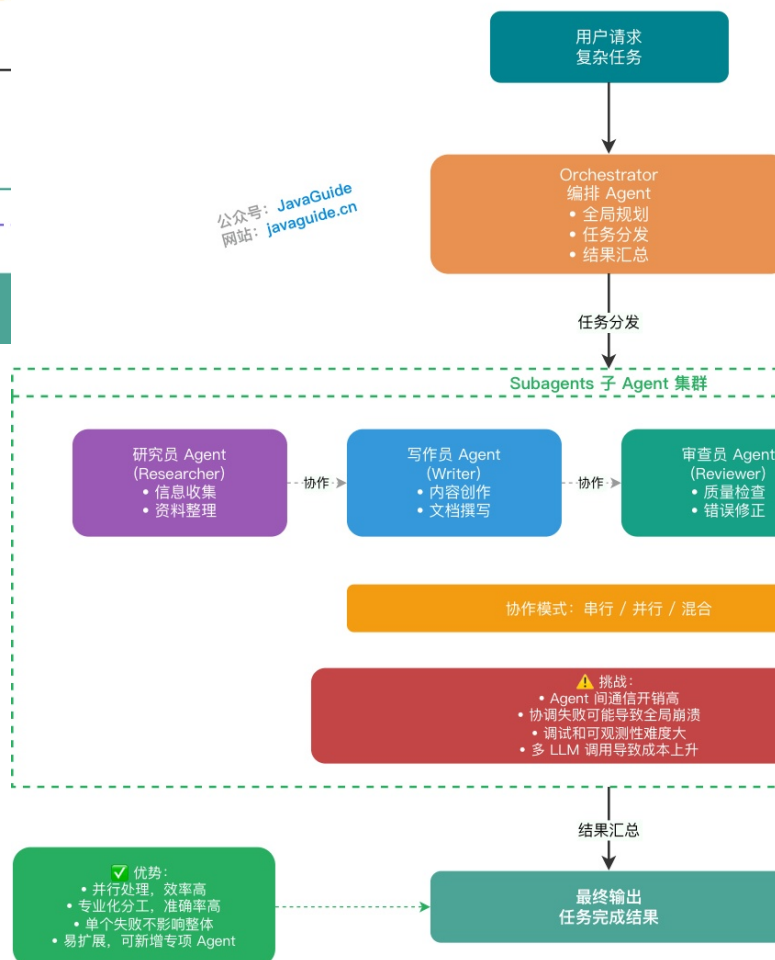
Reflection 很少单独用。更多时候，它会叠加在 ReAct 或 Plan-and-Execute 上，让 Agent 有一定自适应能力。

Multi-Agent

Multi-Agent 是多个独立 Agent 协作完成复杂任务。每个 Agent 专注于一个角色或职能，有点像人类团队分工。常见模式有两种：

1. **Orchestrator-Subagent 模式**：这是现在比较主流的形式。编排 Agent 负责全局规划和任务分发，子 Agent 并行或串行执行具体任务，最后汇总输出。
2. **Peer-to-Peer 模式**：Agent 之间平等对话，互相审查，适合需要辩论、评审、验证的任务。

Multi-Agent 系统架构 (Orchestrator-Subagent)



Multi-Agent 系统架构

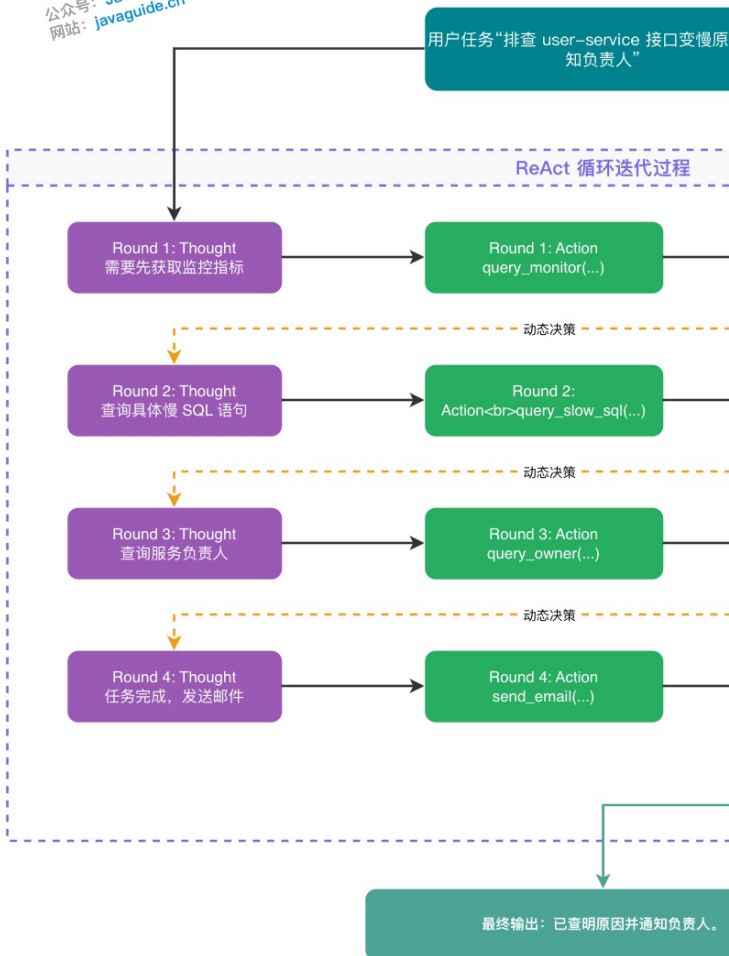
Multi-Agent 的优势是并行效率高，分工更专业，单个 Agent 失败不一定影响整体，也更容易扩展。

问题也很明显：通信成本高，协调失败可能拖垮全局，调试难度大，Token 成本也会上去。

A2A 协议

单个 Agent 升级到 Multi-Agent 后，Agent 之间怎么沟通会变成一个问题。

公众号: JavaGuide
网站: javaguide.cn



ReAct 模式流程

ReAct 的好处是能减少幻觉，复杂任务成功率更高，也比较容易解释每一步为什么这么做。

代价也明显：多轮迭代会增加响应延迟，效果还很依赖工具和 Skills 的质量。

在成熟的 Agent 系统里，查监控、查日志、分析瓶颈这三步可以封装成一个 `diagnose_service_performance` Skill。LLM 只要调用这个 Skill，就能拿到结构化诊断摘要，不用每次都从原子步骤拆起。

Plan-and-Execute

Plan-and-Execute 是 LangChain 团队在 2023 年提出的模式。

它的做法是先让 LLM 制定全局分步计划，再由执行器按步骤完成。

它适合步骤多、依赖关系明确的长期任务。相比 ReAct 边想边做，它更不容易在长任务里迷路。

但它也有问题。计划一旦定下来，执行过程里的动态调整和容错会弱一些，更接近静态 workflow。

实际项目里，两种模式可以组合。

先用 CoT 生成全局步骤，再在每个步骤内部嵌入 ReAct 子循环。这样既有全局结构，也保留局部灵活性。

Reflection

Reflection 给 Agent 加上自我纠错能力。

它不改模型权重，靠自然语言反馈来强化模型行为。

常见实现有三种：

- **Reflexion 框架**：任务失败后进行口头反思，把结论存进记忆缓冲区，下次再遇到类似问题时参考。比如代

如果还靠自然语言互相聊天，Token 消耗很高，也容易出现格式解析错误。

A2A 协议就是为了解决这个问题。

它让 Agent 之间用结构化数据交互，比如带 Schema 的 JSON、XML，或者状态流转指令，而不是一堆自然语言废话。

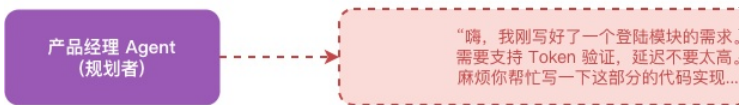
类比一下，后端微服务之间不会通过解析 HTML 页面交换数据，而是用 RESTful 或 RPC 接口传结构化对象。

A2A 协议就是给 Agent 之间定义接口契约。

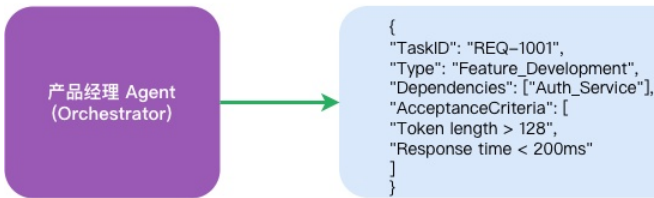
比如“产品经理 Agent”写完需求后，不会输出一句“我写好了，你开发一下”。它应该输出一个标准 JSON Payload，里面包含 TaskID、Dependencies、AcceptanceCriteria。开发 Agent 拿到后直接反序列化，进入执行流程。

A2A (Agent-to-Agent) 通信协议架构

❌ 传统模式：自然语言交互（易幻觉、Token 消耗大、极易解析失败）



✅ A2A 模式：结构化数据交互（确定性强、低损耗、完美契合代码工程）



A2A 协议架构

Agentic Workflows

Agentic Workflows 是吴恩达 (Andrew Ng) 最近重点倡导的概念，可以把前面这些范式放到一起看。

他的观点很务实：没必要一直干等底层模型突破。用工程方法，把推理、工具、记忆、反思、多实体协作编排成流水线，已经能做出很多可用的 AI 应用。

Agentic Workflows (智能体工作流) 迭代模式



智能体工作流核心模式

常见的设计模式包括：

1. Reflection——让模型检查自己的工作
2. Tool Use——给 LLM 配网络搜索、代码执行等工具
3. Planning——让模型提出多步计划并执行

4. Multi-agent Collaboration——多个 Agent 协作完成任务

真实项目里，这几个模式很少单独出现。更常见的是混着用。

比如先 Planning 拆任务，再用 ReAct 执行子任务，中间调用 Tools，最后用 Reflection 做检查。这样看，Agentic Workflows 更像是一套工程组合拳，而不是某个单独框架。

AI 工作流和 Agent 到底是什么关系？

前面一直在说“工作流”，但如果不把它和 Agent 的区别讲清楚，后面选型很容易乱。

很多人一听 Agent，就默认应该让模型自己规划、自己调用工具、自己跑完全程。听起来很智能，实际落地不一定稳。

纯 Agent 里，LLM 是决策者。每一步要不要调工具、调哪个工具、下一步怎么走，主要靠模型推理。你给它一个任务，它自己尝试把任务跑完。

AI 工作流里，LLM 只是流程里的一个节点。整条流程的骨架，比如步骤顺序、条件跳转、失败重试，都是你提前设计好的。控制权在图结构里，不在模型手里。

Agentic Workflows 则是两者混着用：全局用 Workflow 管住结构，在某些不确定的节点里嵌入 Agent 子循环，让模型自己探索一小段。

工作流里的 Node、Edge、State 是什么？

AI 工作流的数据结构是有向图 (Graph)，三个元素：Node (节点) 负责执行，Edge (边) 负责控制流，State (状态) 在节点之间共享上下文。

Node 只做一件事，读取状态、执行逻辑、写回结果。节点里可以调 LLM，可以是工具调用，也可以是纯代码逻辑。写文章这个场景里，典型节点是“生成初稿”“质量审核”“按反馈修改”，节点职责越单一，越容易排查。Edge 决定执行完跳到哪——顺序边按路径走，条件边根据运行时状态分支，循环边让流程回到之前的节点重试。State 记录当前草稿、评分、重试次数这类东西，条件边的跳转往往基于 State 里的值来判断。

“审核不通过就回到修改，最多重试 3 次”，翻译成图结构，是一条从 ReviewNode 指向 ReviseNode 的条件边，加上 `iteration_count >= 3` 时跳到 ExitNode 的安全边界。State 里的 `iteration_count` 是让这条逻辑能跑起来的关键。

迭代模式结构比写死的 if-else 链更容易扩展，出了问题也好定位到哪个节点哪条边。LangGraph (Python) 和 Spring AI Alibaba Graph (Java) 都是基于这套思路实现的。详细设计和代码实现可以看：《AI 工作流中的 Workflow、Graph 与 Loop》。

什么时候用 Agent，什么时候用 Workflow？

执行路径能不能提前确定，是最简单的判断标准。

能确定，用 Workflow。不能确定，用 Agent。两者都有，用 Agentic Workflows。

但有个常见认知偏差：很多人觉得任务“路径不确定”，其实是需求没拆清楚。把任务认真拆一遍后，往往会发现大部分场景是“LLM 在固定节点里做生成或判断”，这种用 Workflow 更稳，也更容易排查。

真正适合纯 Agent 的任务，是那种你提前写不出执行步骤的场景。比如“帮我排查这个线上故障”，查什么、怎么查、查到什么程度，很难事先规定死。

另一个判断维度是容错要求。Workflow 执行路径固定，出问题好排查；Agent 执行路径动态，调试难度高一个数量级。To B 商业场景优先考虑 Workflow 或 Agentic Workflows。

各范式怎么选？

前面讲了 ReAct、Plan-and-Execute、Reflection、Multi-Agent、AI 工作流这一堆概念，做项目时面对这些选型容易头大。做个简单的参考：

场景特征	推荐方向	代价
执行路径可提前确定，节点需要 LLM	AI 工作流 (Graph)	稳定可观测，前期设计成本高
执行路径不确定，需要动态规划	ReAct	灵活，Token 消耗高，调试难
任务很长，步骤多但结构清晰	Plan-and-Execute	不易迷路，动态调整弱
输出质量要求高，允许多轮迭代	叠加 Reflection E 配合用，不单独用	和 ReAct/P
任务天然可拆成多个专业角色	Multi-Agent	通信和调试成本翻倍
长任务 + 部分子任务不可预测	Agentic Workflows	全局 Workflow + 局部 ReAct 嵌套

先用最简单的方式跑通，再根据实际失败模式决定升级哪一层。

上来就搞 Multi-Agent、全靠模型动态推理、上下文不做任何管理，踩进去了再爬出来会很费劲。

总结

大部分 Agent 项目跑起来不稳定，不是模型不够好。基础没搭好。LLM + Planning + Memory + Tools 四块，缺哪个都有明显短板。Tools 没有，Agent 停留在“给建议”阶段；Memory 没有，稍微长一点的任务就开始失忆；上下文管不好，模型随便跑偏。

选型也容易选错。ReAct 灵活但调试难，Token 烧得也多；Workflow 稳但对需求拆解要求高，提前设计不够充分的话，后面改起来也费劲；Multi-Agent 接入后通信和调试成本容易超出预期。上来就搞最复杂的方案，是工程实践里最常见的陷阱。

还有一块很容易忽略：工具描述。MCP 解决接入方式，JSON Schema 解决描述格式，但模型到底调不调这个工具、参数怎么填，最后都靠 description 里那几句话。这块省了力气，后面会双倍还回来。

Agent 和工作流的选型其实没那么复杂，先把任务执行路径写出来，能写出来就用 Workflow，写不出来再上 Agent。这个判断先做好，比追框架有用得多。